PRACTICAL 14

Admin Dashboard with Role-Based Access Control, Dynamic Menu, and Audit Log

AIM

To create a database-driven admin dashboard with user, student, event and registration statistics, role-appropriate navigation, recent registrations and an audit trail.

Introduction

The admin dashboard queries current counts and latest event registrations from MySQL. Admin navigation links expose student management, event management and the Practical 13 API demo. A general audit_logs table records important student and event management actions without storing passwords, session tokens or form contents.

Tools and Technology

VS Code, XAMPP, PHP, MySQLi, MariaDB, HTML, CSS, JavaScript Fetch API, browser, Git and GitHub.

Files Used

File	Purpose
admin-dashboard.php	Admin-only statistics, registration list and recent audit activity.
php/admin-audit.php	Shared helper for recording administrative changes.
php/event-functions.php	Adds event actions to the general audit trail.
students-manage.php	Records add, edit, soft-delete and restore actions.
sql/practical14.sql	Additive audit_logs table migration.
css/style.css	Responsive dashboard statistics grid.
scripts/test-practical14.py	Repeatable role, dashboard and menu checks.
docs/Practical14-setup.md	Migration and role verification steps.

Key Skills Used

Dashboard design, role-based access control, SQL aggregation, prepared queries, audit logging, role-based navigation and server-side authorization.

Implementation Overview

1 Database Statistics

Four live counts are displayed for users, active students, events and event registrations. Counts are calculated with SQL against the current database.

2 Latest Registrations

The dashboard joins registrations to student and event records and shows the five most recent entries with their registration time.

3 Admin-Only Access and Menu

The page calls requireRole(admin) before loading dashboard data. Management links are present on the admin dashboard; faculty and student dashboards remain separately protected.

4 General Audit Log

The migration creates audit_logs with actor, action, target type, target ID, summary and timestamp. Student add/edit/delete/restore and event add/edit/delete operations write entries.

5 Safe Audit Data

Audit summaries include record names or IDs for review. They deliberately exclude password values, CSRF tokens, remember-me validators and complete submitted forms.

Implementation Evidence

The additive audit_logs migration is installed. Anonymous dashboard access redirects to login. Signed-in admin checks pass for the dashboard, four statistic cards, latest registration section, audit section and management/API navigation. Student and faculty menu checks remain pending their existing passwords.

Database Results

Check	Observed result
Dashboard counts	Users, active students, events and registrations.
Recent registrations	Latest five registration rows joined to student and event titles.
Audit retention	Audit entries remain after a target event is deleted.
Migration	Additive CREATE TABLE IF NOT EXISTS; existing data is preserved.

Testing Summary

Test input or action	Observed result
Audit table migration	PASS: audit_logs created without changing existing tables.
Anonymous admin dashboard access	PASS: redirects to login.
Anonymous API demo access	PASS: redirects to login.
Admin dashboard, four cards, registrations, audit and menu	PASS: all 5 admin checks.
Student / faculty dashboard and role menus	Pending: their existing passwords were not changed.
Audit entries from student/event CRUD	Hooks are implemented; authenticated record-change checks remain pending.

Screenshots of Portal Implementation

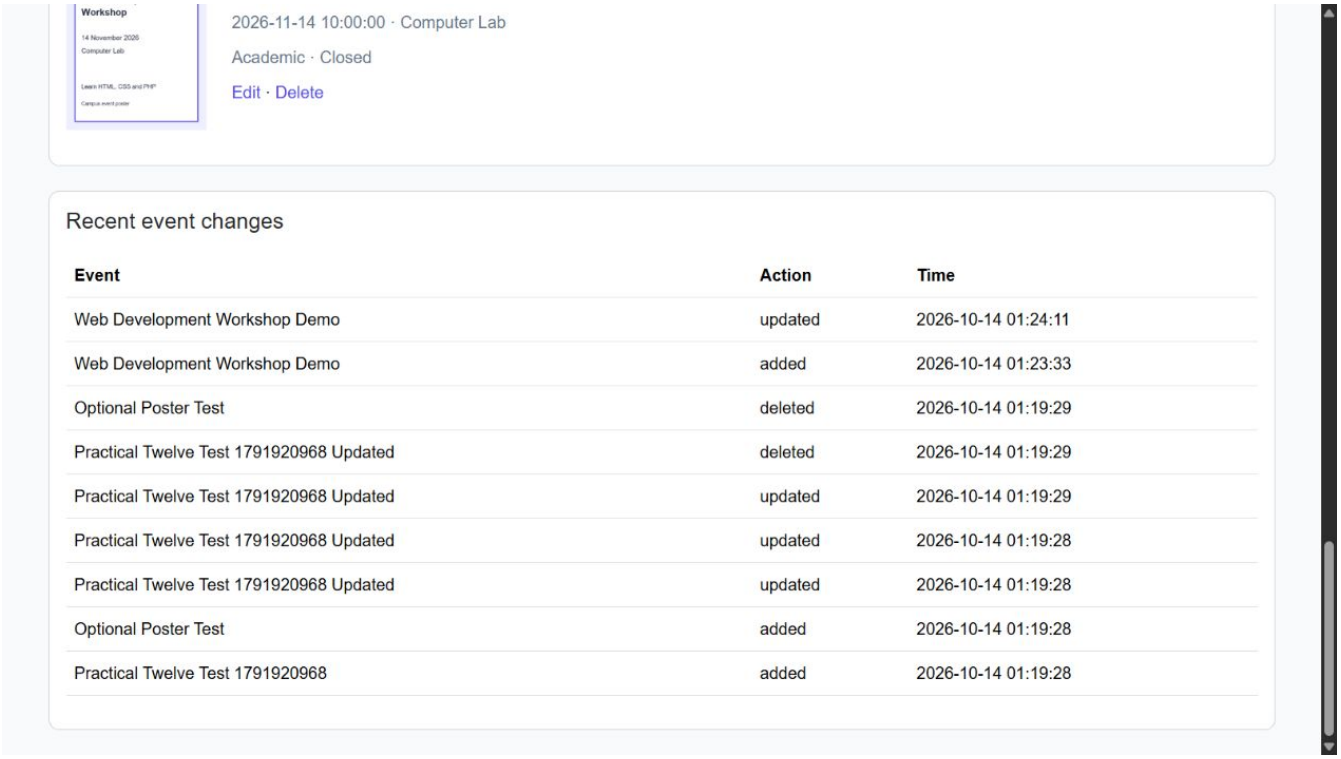


Figure 1: Existing Practical 12 event-change audit view; the new Practical 14 audit_logs dashboard view still needs a fresh capture.

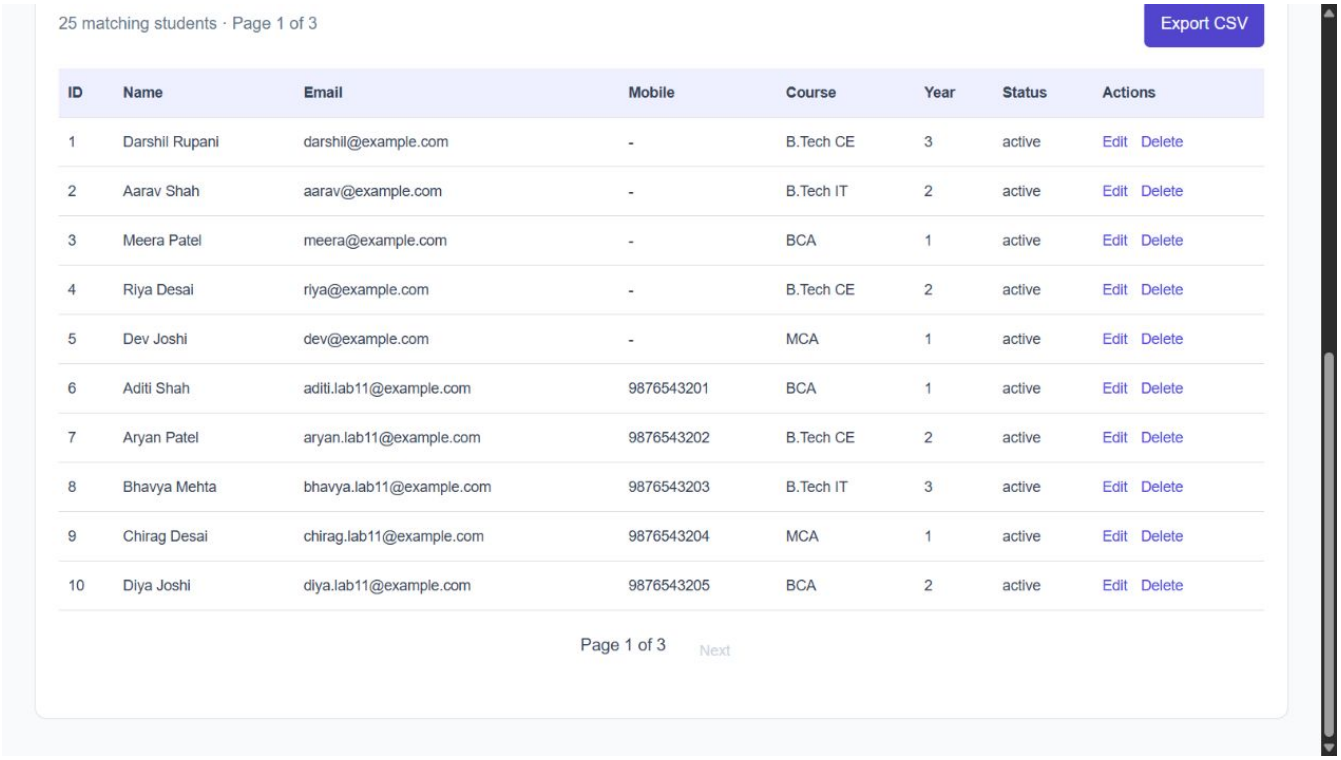


Figure 2: Existing protected student-management listing used as role-based admin UI context.

Key Questions and Answers

Q1 Are dashboard values loaded dynamically from the database

Yes. Counts and registration rows are queried for each admin dashboard request.

Q2 Is access restricted to admin users

Yes. requireRole checks the current database role before the page runs its dashboard queries.

Q3 Are role-based menus working

The management and API navigation is shown on the admin dashboard. Student and faculty pages remain separate role-protected destinations.

Q4 Are important admin actions recorded in audit_logs

Student record changes and event CRUD actions call the shared audit helper. The migration stores the actor, action, target and time.

GitHub Repository

https://github.com/DarshilRupani/studenthub-portal
Practical 13–15 files and reports are committed and pushed to the main branch.

Conclusion

The audit migration, anonymous access gate and signed-in administrator dashboard checks pass. Student and faculty navigation checks still need their existing lab passwords; event/student audit-write behavior remains to be checked with authenticated CRUD.